

CprE 3810, Computer Organization and Assembly Level Programming

Lab 1 Report

Student Name Ryan Hoefer_____

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 3.1.c] Think of three more cases and record them in your lab report.

Case 3: perform mac operation to achieve oY of 29
with the wight initializatoin at 10, set activation input of iX to to 2 and the partial sum input to iY to 9.

the wight is not changed with iLdW being set to 0. so oY is $29 = 10 * 2 + 9$

Case 4: perform the mac operation with a new weight.

Set the iLdw to 1 and iW to 5. Keep same ix value of 2 and iY value of 9.

oY will be $19 = 5 * 2 + 9$

Case 5: perform Mac operation with iY of 0

set the iLdw value to 0 to keep the wight the same. Then keep same ix value of 2 and iY value of 9. to get a final oY value of $10 = 5 * 2 + 0$

[Part 3.1.e] For labels 1, 7, 22, and 28, specify where (VHDL file and line number) these values are located – some will be found in more than one place. Also attempt to explain the functionality of each label as it occurs in the code

1) in the TPU_MV_ELEMENT.vhd file, it is declared on line 24 with the inputs and outputs being defined below lines.

7) in the same TPU_MV_ELEMENT.vhd file on line 131, the d_Add1 module is efined. This will calculate the output for oY

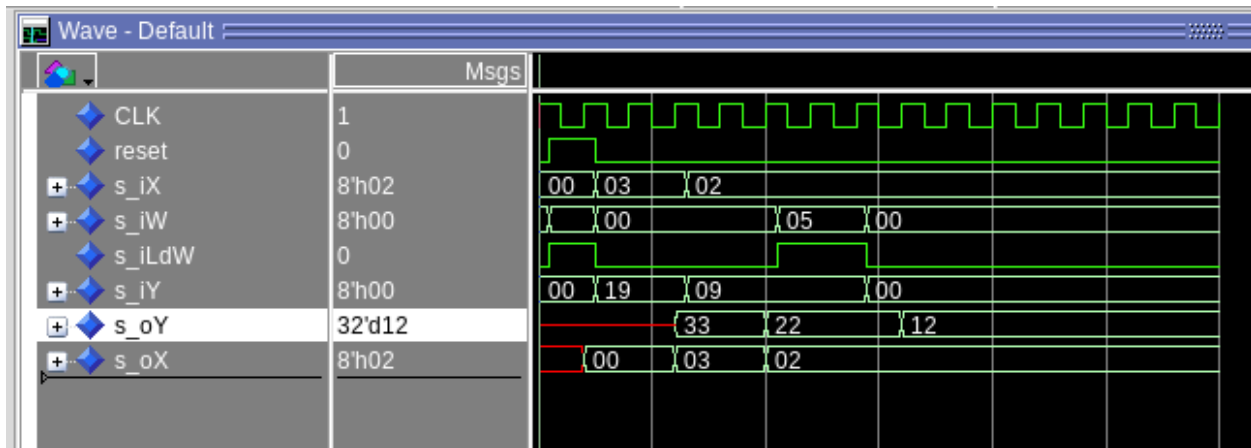
22) oQ is assigned at line 50 of the regLd.vhd. Will change the value of sQ if iLd is 1. In t he TPU_MV_ELEMENT.vhd file it is assigned to s_W.

28) iD is used on line 50 of file Reg.vhd. Its value is directly assigned to the output value on a rising clock edge. On line of 128 of the TPU_MV_ELEMENT.vhd file it is assigned to s_X1 From the g_Delay1 file.

[Part 3.1.f] Implement your test cases from part c into this testbench. Remember to include these in your waveform screenshot.

```
File Edit Selection View Go Run ... Search Update
tb_TPU_MV_Element(TestCasesAdded).vhd tb_TPU_MV_Element(addedtests).vhd Welcome TPU_MV_Element.vhd Multiplier.vhd tb ...

home > rhoefier > cpre3810 > Lab1 > Lab1 > tb_TPU_MV_Element(addedtests).vhd
59 begin
104 begin
127 -- TODO: add test cases as needed (at least 3 more for this lab)
128 -- Test case 3:
129 --achieve mac operations to get oY of 29
130 s_iX <= x"02"; -- 2 in hexadecimal
131 s_iW <= x"00"; -- Not strictly necessary, but this makes the testcases easier to read
132 s_iLdW <= '0'; -- Make sure we don't continue to load.
133 s_iY <= x"09"; -- 9 in hexadecimal
134 wait for gCLK_HPER*2;
135 wait for gCLK_HPER*2;
136 -- expect oY to be 29 and o_x to be 2
137
138 -- Test case 4:
139 s_iX <= x"02"; -- 2 in hexadecimal
140 s_iW <= x"05";
141 s_iLdW <= '1';
142 s_iY <= x"09"; -- 9 in hexadecimal
143 wait for gCLK_HPER*2;
144 wait for gCLK_HPER*2;
145 --expect o_Y to be 19 and o_x to be 2
146
147 -- Test case 5:
148 s_iX <= x"02"; -- 2 in hexadecimal
149 s_iW <= x"00"; -- Not strictly necessary, but this makes the testcases easier to read
150 s_iLdW <= '0'; -- Make sure we don't continue to load.
151 s_iY <= x"00";
152 wait for gCLK_HPER*2;
153 wait for gCLK_HPER*2;
154 --expect o_y to be 10 and o_X to be 2
155
156
157
158
159 wait;
160 end process;
```



The testbench code is on the top with the waveform on the bottom

[Part 3.1.g.v] In your lab report, include a screenshot of the waveform. Describe, in plain English, any differences between what you expected and what the simulation showed.

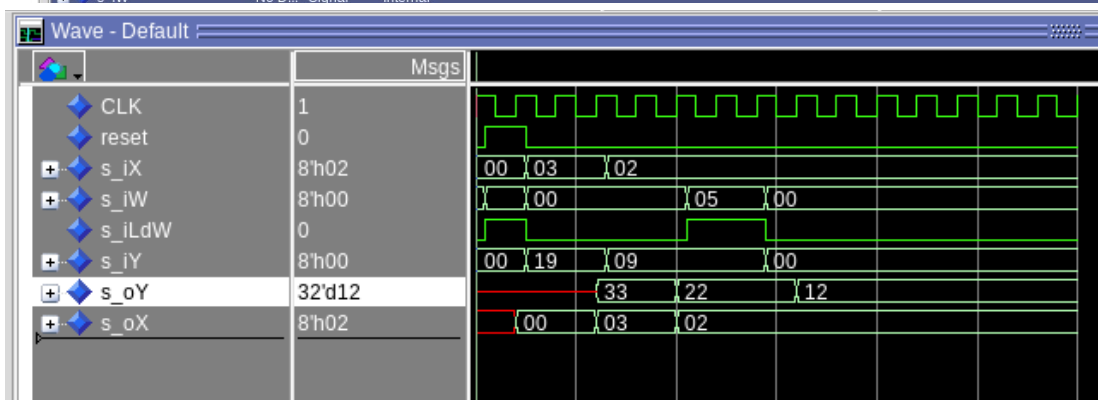
```

File Edit Selection View Go Run ... Search Update
tb_TPU_MV_Element(TestCasesAdded).vhd tb_TPU_MV_Element(addedtests).vhd Welcome TPU_MV_Element.vhd Multiplier.vhd tb

home > rhoefier > cpre3810 > Lab1 > Lab1 > Lab1 > tb_TPU_MV_Element(addedtests).vhd
59 begin
104 begin
127 -- TODO: add test cases as needed (at least 3 more for this lab)
128 -- Test case 3:
129 --achieve mac operations to get oY of 29
130 s_iX <= x"02"; -- 2 in hexadecimal
131 s_iW <= x"00"; -- Not strictly necessary, but this makes the testcases easier to read
132 s_iLdW <= '0'; -- Make sure we don't continue to load.
133 s_iY <= x"09"; -- 9 in hexadecimal
134 wait for gCLK_HPER*2;
135 wait for gCLK_HPER*2;
136 -- expect oY to be 29 and o_x to be 2
137
138 -- Test case 4:
139 s_iX <= x"02"; -- 2 in hexadecimal
140 s_iW <= x"05";
141 s_iLdW <= '1';
142 s_iY <= x"09"; -- 9 in hexadecimal
143 wait for gCLK_HPER*2;
144 wait for gCLK_HPER*2;
145 --expect o_Y to be 19 and o_x to be 2
146
147 -- Test case 5:
148 s_iX <= x"02"; -- 2 in hexadecimal
149 s_iW <= x"00"; -- Not strictly necessary, but this makes the testcases easier to read
150 s_iLdW <= '0'; -- Make sure we don't continue to load.
151 s_iY <= x"00";
152 wait for gCLK_HPER*2;
153 wait for gCLK_HPER*2;
154 --expect o_y to be 10 and o_X to be 2
155
156
157
158
159 wait;
160 end process;

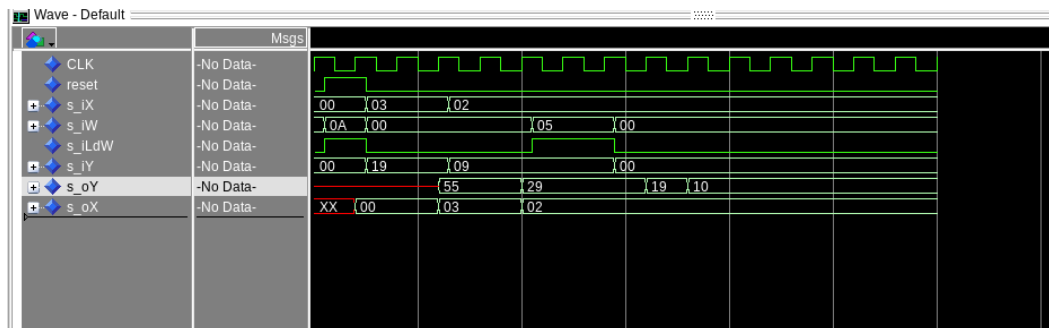
```

Ln 154, Col 42 Spaces: 2 UTF-8 LF {} Plain Text Sign In



The O_Y outputs are not as expected, for test 2 it is ox21(33) instead of 0x37(55), for test 3 it should be ox1D(29) but is ox16(22), for test 4 it should be ox13(19) but is 0xC(12) , and for 5 it should be 0xA(10) but is also 0xC(12)

[Part 3.1.h] In your lab report, include a screenshot of the waveform. Describe, in plain English, how your waveform matches the expected result (e.g., reference the specific cycles and times). In your submission zip file, provide the completed *TPU_MV_Element.vhd* file in a folder called 'MAC'.

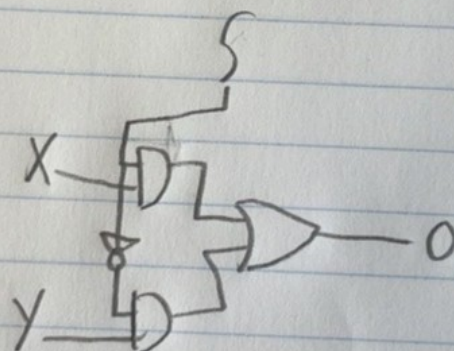


OY has nothing until the first positive edge as expected the nit switches to the hex value 0x37 as expected. Then two cycles later changes to 0x1D as expected. Then 3 cycles later changes to 0x13, it is 3 cycles instead of two because the load is changing so it takes an extra cycle to apply. Then 1 cycle later changes to 0xA, only 1 cycle because of the deley of the load changing.

[Part 3.3.a] Draw the truth table, Boolean equation, and Boolean circuit equivalent (using only two-input gates) that implements a 2:1 mux. Include this in your lab report.

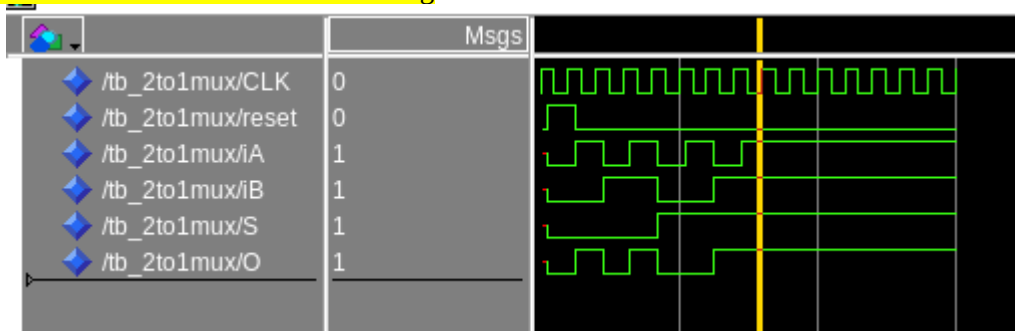
S	X	Y	O
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

S	X	Y	O
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



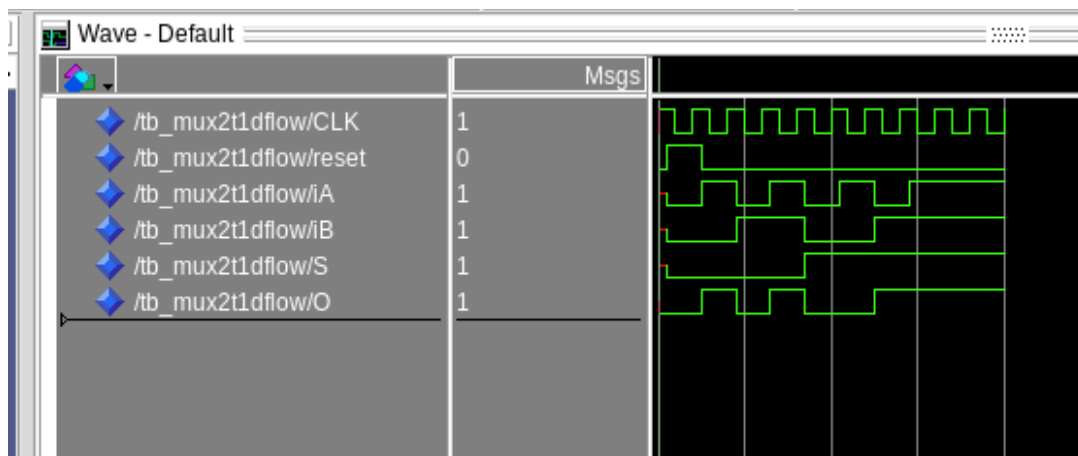
$$(X\bar{S}) + (XS) = O$$

[Part 3.3.d] In your lab report, include a screenshot of the waveform. Make sure to label the screenshot with which module it is testing.



This is a waveform of a 2 to 1 mux. Module name 2to1mux.

[Part 3.3.e] Again, in your lab report, include a labeled screenshot of the waveform showing the dataflow mux implementation working.



This is the 2t1 mux using dataflow. The module is mux2t1dflow

[Part 3.4] Include a waveform screenshot and corresponding description demonstrating it is working correctly.



This is working as expected, for first test, both are 0 so we get 0, for second, A is FF and is selected so out put is FF. For third, B is selected and out putted to FF. For fourth, both are FF and outputs FF. For fifth, A is 23 and selected and output is 23.

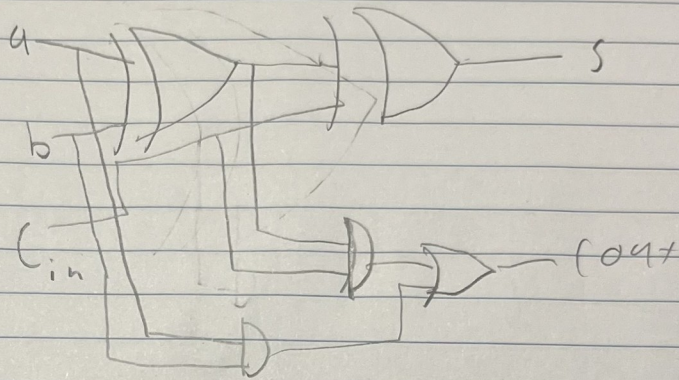
[Part 3.5.b] Include a waveform screenshot and description in your lab report.

		Msgs						
	/tb_onescomp_n/iA	32'h5555AAAA	00000000	FFFFFFF	FFFF0000	0000FFFF	5555AAAA	
	/tb_onescomp_n/O	32'hAAAA5555	FFFFFFF	00000000	0000FFFF	FFFF0000	AAAA5555	

the input nots the output, iA is the input line an O is the output line.

[Part 3.6.a] A full adder takes three single-bit inputs and produces two single-bit outputs – a sum and carry for the addition of the three input bits. Draw the truth table, Boolean equation, and Boolean circuit equivalent (using only two-input gates) that implements a 1-bit full adder. Include this in your report.

3.60



a	b	c_{in}	S	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

a \ b	00	01	11	10
0	0	1	0	1
1	1	0	1	0

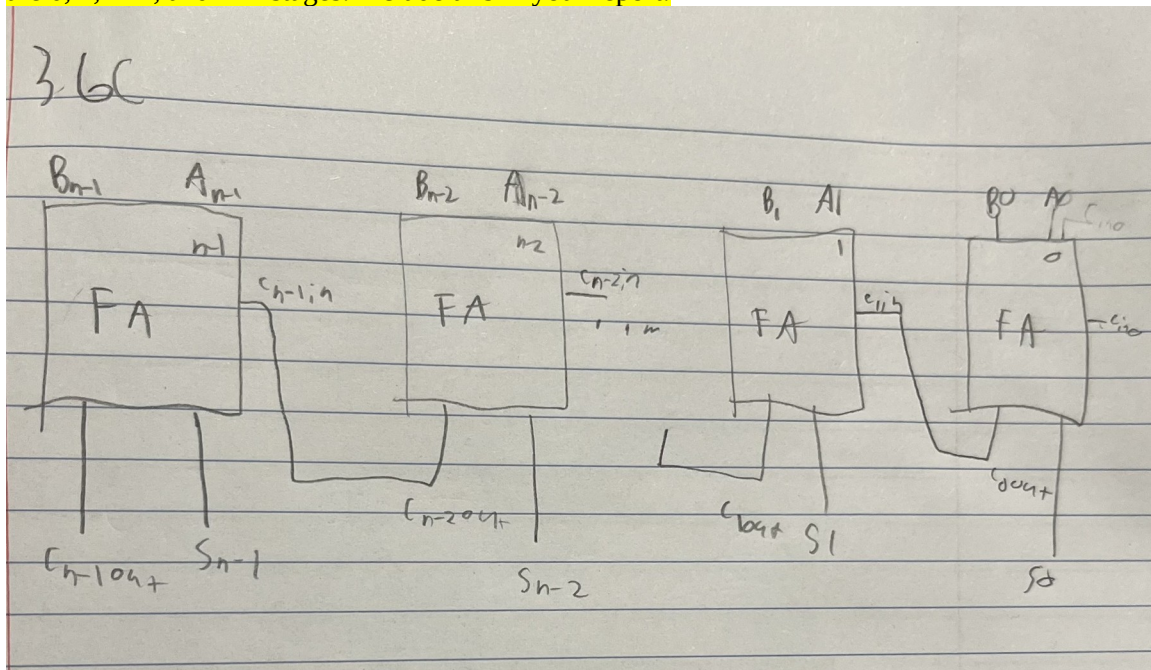
a \ b	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$S = (a \oplus b) \oplus c_{in}$$

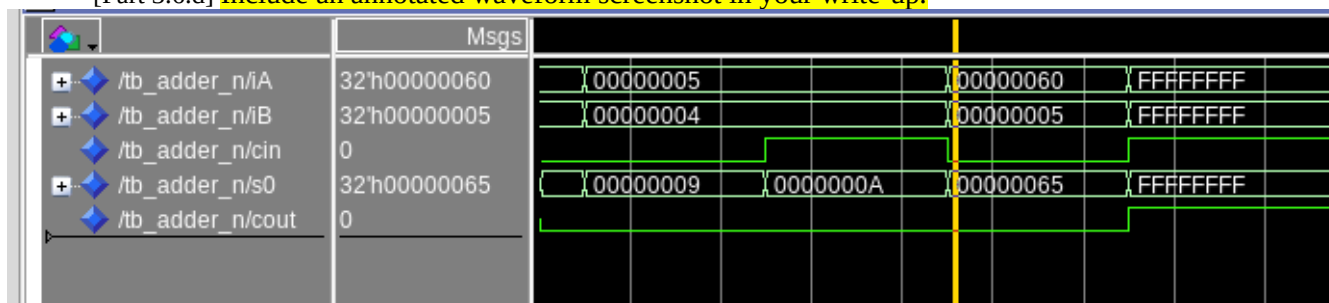
$$c_{out} = (ab) + (bc) + (ac)$$

$$c_{out} = (S, c_{in}) + (ab)$$

[Part 3.6.c] Then draw a schematic of the intended design, including inputs and outputs and at least the 0, 1, N-2, and N-1 stages. Include this in your report.



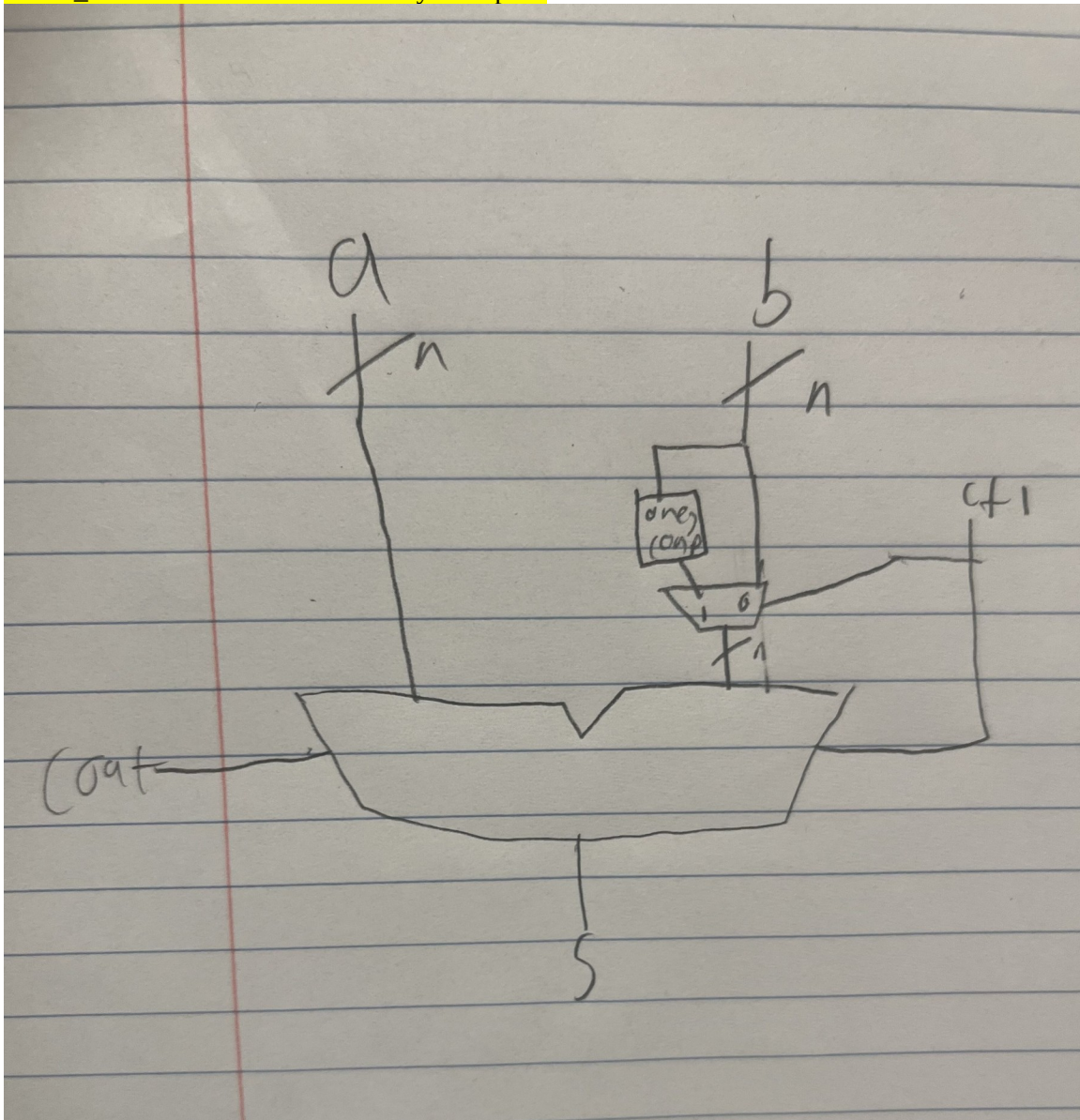
[Part 3.6.d] Include an annotated waveform screenshot in your write-up.



This correctly adds both iA and iB plus the cin to get the sum. The second test shows the carry in works as expected and the final test shows the c out works as expected.

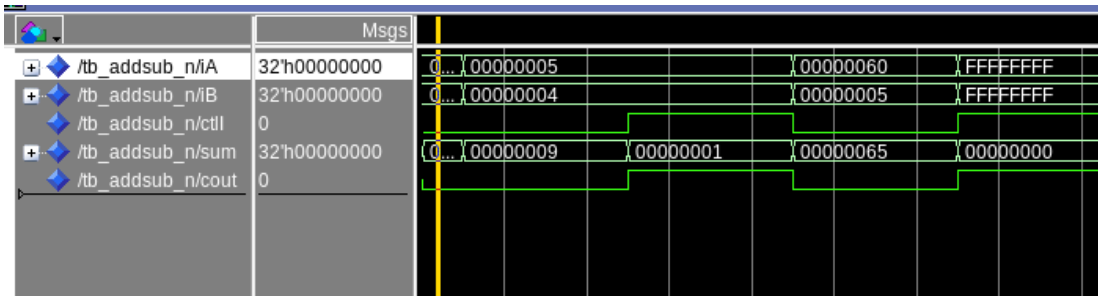
[Part 3.7.a] Draw a schematic (don't use a schematic capture tool) showing how an N-bit adder/subtractor with control can be implemented using only the three main components designed

in earlier parts of this lab (i.e., the N-bit inverter, N-bit 2:1 mux, and N-bit adder). How is the 'nAdd_Sub' bit used? Include this in your report.

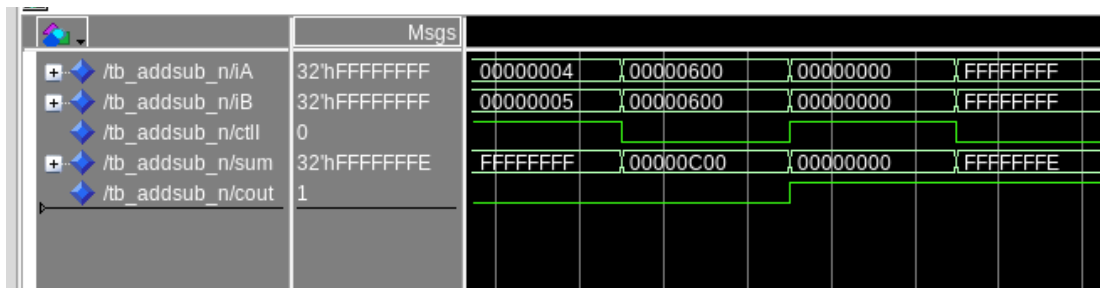


The 'nAdd_Sub' bit is used to select which version of b will be added to A, (B or Not B).

[Part 3.7.c] Provide multiple waveform screenshots in your write-up to confirm that this component is working correctly. What test-cases did you include and why?



- 1) $5+4 = 9$ – to test basic addition
- 2) $5-4 = 1$ – to test basic subtraction
- 3) $0x60 + 0x5 = 65$ to test larger addition
- 4) $0xFFFFFFFF - 0xFFFFFFFF = 0$ – to test large edge case.



F

- 1) $4 - 5 = -1$ ($0xFFFFFFFF$ in two's comp) to test subtraction rolling over (cout is 0)
- 2) $0x600 + 0x600 = 0xC00$ to test larger addition
- 3) $0 - 0 = 0$ to test 0 extreme case
- 4) $0xFFFFFFFF + 0xFFFFFFFF = 0x1FFFFFFFE$ to test c out carry for large input